

Dr > NewwwwWW > ComputerOrg > Lab06 > cache > cachesrcs > sources_1 > new > Data_ram0.v

```

5  (
6
7  );
8
9  output reg [Block_width-1:0] dout // data read out
10 );
11
12
13 parameter NUM_of_sets = 2 ** Index_width;
14
15 reg [Block_width-1:0] cache_data [0:NUM_of_sets-1];
16 reg [Block_width-1:0] same_data;
17
18 // initial begin
19 // $readmemh("data_ram0.mem", cache_data);
20 // end
21
22 initial begin
23     cache_data[0] = 128'h00000001_00000002_00000003_00000004;
24     cache_data[1] = 128'h10000001_10000002_10000003_10000004;
25     cache_data[2] = 128'h20000001_20000002_20000003_20000004;
26     cache_data[3] = 128'h30000001_30000002_30000003_30000004;
27 end
28
29 always @(posedge clk)
30 begin
31     same_data <= cache_data[addr];
32     if (en) begin
33         cache_data[addr] <= din;
34     end
35     dout <= same_data;
36 end
37 endmodule

```

Tag_ram0.v X

Dr > NewwwwWW > ComputerOrg > Lab06 > cache > cachesrcs > sources_1 > new > Tag_ram0.v

```

14 output reg [Block_width-1:0] dout // data read out
15 );
16
17 parameter NUM_of_sets = 2 ** Index_width;
18
19 reg [Block_width-1:0] cache_tag [0:NUM_of_sets-1];
20 reg [Block_width-1:0] same_data;
21
22 // initial begin
23 // $readmemh("tag_ram0.mem", cache_tag);
24 // end
25
26 initial begin
27     cache_tag[0] = 26'b10000000000000000000000000000000;
28     cache_tag[1] = 26'b10000000000000000000000000000001;
29     cache_tag[2] = 26'b10000000000000000000000000000010;
30     cache_tag[3] = 26'b10000000000000000000000000000011;
31 end
32
33 always @(posedge clk)
34 begin
35     same_data <= cache_tag[addr];
36     if (en) begin
37         cache_tag[addr] <= din;
38     end
39     dout <= same_data;
40 end
41 endmodule

```

Tag_ram0.v X

Dr > NewwwwWW > ComputerOrg > Lab06 > cache > cachesrcs > sources_1 > new > Data_ram1.v

```

11 );
12
13 parameter NUM_of_sets = 2 ** Index_width;
14
15 reg [Block_width-1:0] cache_data [0:NUM_of_sets-1];
16 reg [Block_width-1:0] same_data;
17
18 // initial begin
19 // $readmemh("data_ram1.mem", cache_data);
20 // end
21
22 initial begin
23     cache_data[0] = 128'h0000000a_0000000b_0000000c_0000000d;
24     cache_data[1] = 128'h1000000a_1000000b_1000000c_1000000d;
25     cache_data[2] = 128'h2000000a_2000000b_2000000c_2000000d;
26     cache_data[3] = 128'h3000000a_3000000b_3000000c_3000000d;
27 end
28
29 always @(posedge clk)
30 begin
31     same_data <= cache_data[addr];
32     if (en) begin
33         cache_data[addr] <= din;
34     end
35     dout <= same_data;
36 end
37 endmodule

```

Tag_ram1.v X

Dr > NewwwwWW > ComputerOrg > Lab06 > cache > cachesrcs > sources_1 > new > Tag_ram1.v

```

15 );
16
17 reg [Block_width-1:0] cache_tag [0:NUM_of_sets-1];
18 reg [Block_width-1:0] same_data;
19
20 // initial begin
21 // $readmemh("tag_ram1.mem", cache_tag);
22 // end
23
24 initial begin
25     cache_tag[0] = 26'b10000000000000000000000000000000;
26     cache_tag[1] = 26'b10000000000000000000000000000001;
27     cache_tag[2] = 26'b10000000000000000000000000000010;
28     cache_tag[3] = 26'b11100000000000000000000000000011; // valid but [LRU(old) + dirty]
29 end
30
31 always @(posedge clk)
32 begin
33     same_data <= cache_tag[addr];
34     if (en) begin
35         cache_tag[addr] <= din;
36     end
37     dout <= same_data;
38 end
39 endmodule

```

Tag_ram1.v X

对 cache 中的 data+tag 进行初始化, 左上角为 Data0 (Way0), 左下角为 Tag0 (Way0); 右上角为 Data1 (Way1), 右下角为 Tag1 (Way1) .

·仿真代码

```
`timescale 1ns / 1ps
module Cache_tb();
    reg clk;
    reg rst;
    reg [31:0] addr_cpu;
    reg rd_cpu;
    reg wr_cpu;
    reg ready_mem;

    // Outputs
    wire [31:0] addr_mem;
    wire rd_mem;
    wire wr_mem;

    // Bidirs
    reg [31:0] data_cpu_write;
    reg [127:0] data_mem_read;
    wire [127:0] data_mem_write;
    wire [31:0] data_cpu_read;

    // Instantiate the Unit Under Test (UUT)
    Cache uut (
        .clk(clk),
        .rst(rst),
        .data_cpu_write(data_cpu_write),
        .data_mem_read(data_mem_read),
        .addr_cpu(addr_cpu),
        .addr_mem(addr_mem),
        .rd_cpu(rd_cpu),
        .wr_cpu(wr_cpu),
        .rd_mem(rd_mem),
        .wr_mem(wr_mem),
        .data_cpu_read(data_cpu_read),
        .data_mem_write(data_mem_write),
        .ready_mem(ready_mem)
    );

    //Clock generator block
    initial begin
        clk = 1'd0;
    forever
        #10 clk = ~clk;
    end
end
```

```

initial begin
    // Initialize Inputs
    rst = 1;
    addr_cpu = 0;
    rd_cpu = 0;
    wr_cpu = 0;
    ready_mem = 1;
    data_cpu_write = 0;
    data_mem_read = 0;
    rst = 0;
    #120;

    // Read from location
    rd_cpu = 1'd1;
    addr_cpu = 32'b0000_0000_0000_0000_0000_0010_0000_0111;
    #40;
    rd_cpu = 1'd0;

    // Write to same location
    #40;
    wr_cpu = 1'd1;
    data_cpu_write = 32'habababab;
    addr_cpu = 32'b0000_0000_0000_0000_0000_0010_0000_0111;
    #40;
    wr_cpu = 1'd0;
    // Read from same location to check updated data
    #40;
    rd_cpu = 1'd1;
    addr_cpu = 32'b0000_0000_0000_0000_0000_0010_0000_0111;
    #40;
    rd_cpu = 1'd0;

    // //      Read Miss with dirty bit 0 policy check, reads data from
Main Memory

    #40;
    rd_cpu = 1'd1;
    addr_cpu = 32'b0000_0000_0000_0000_0000_0010_0000_1010; // read
miss
    #40;
    ready_mem = 1;
    data_mem_read = 128'h54088666_54088666_54088666_54088666; //
miss

```

```

        #40;
        rd_cpu = 1'd0;
        #40;
        rd_cpu = 1'd1;
        addr_cpu = 32'b0000_0000_0000_0000_0000_0010_0000_1010;
        #40;
        rd_cpu = 1'd0;

// ////////////////////////////////////////
//      // write Miss with dirty bit 1 policy check
        #80;
        rd_cpu = 1'd0;
        wr_cpu = 1'd1;
        data_cpu_write = 32'h5a5a5a5a;
        addr_cpu = 32'b0000_0000_0000_0000_0000_0010_0000_1101;
        #40;

        ready_mem = 1;
        data_mem_read = 128'h11451450_11451451_11451452_11451453;

        #40;
        wr_cpu = 1'd0;
        #40;
        wr_cpu = 1'd1;
        data_cpu_write = 32'h5a5a5a5a;
        addr_cpu = 32'b0000_0000_0000_0000_0000_0010_0000_1101;
        #40;
        wr_cpu = 1'd0;

        #300 $finish;

    end

endmodule

```

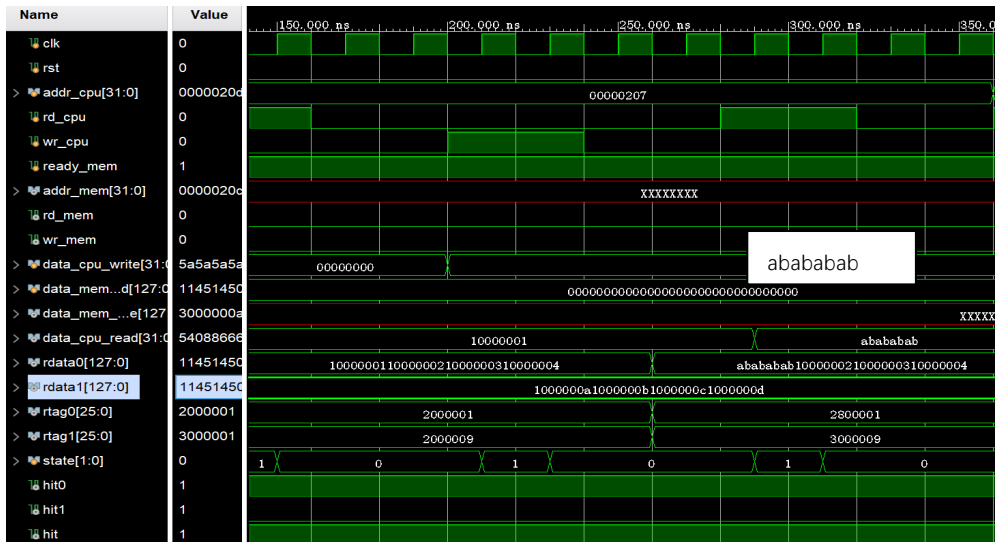
(1) Read Hit



```
addr_cpu = 32'b0000_0000_0000_0000_0000_001_0000001_11; = 207
```

Index = 0000001 第一组 ——> Tag = 000001 和 rtag0 相等(hit) ——> 读出 rdata0 的第 b11 = d3 个字的数据 (10000001)。

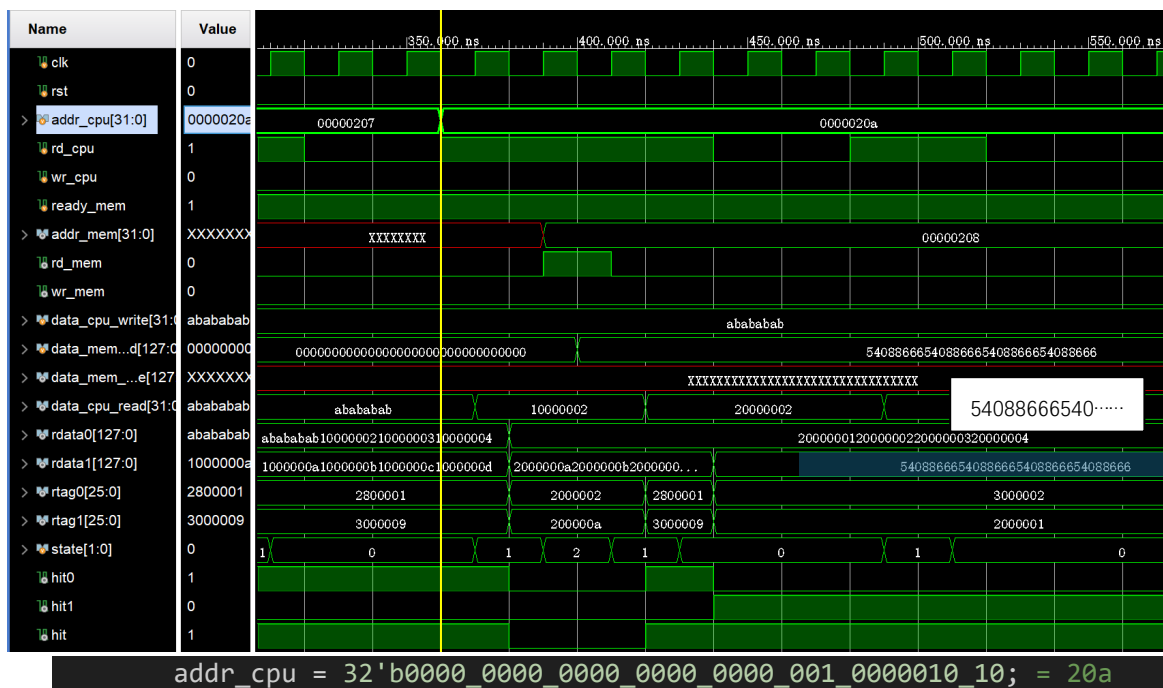
(2) Write hit



同一个 `addr_cpu`, 现在想要写入 `h'abababab`, 可以看到 `rdata0` 中的数据被修改 (Write hit), But (`data_cpu_write` 没被截进来)。

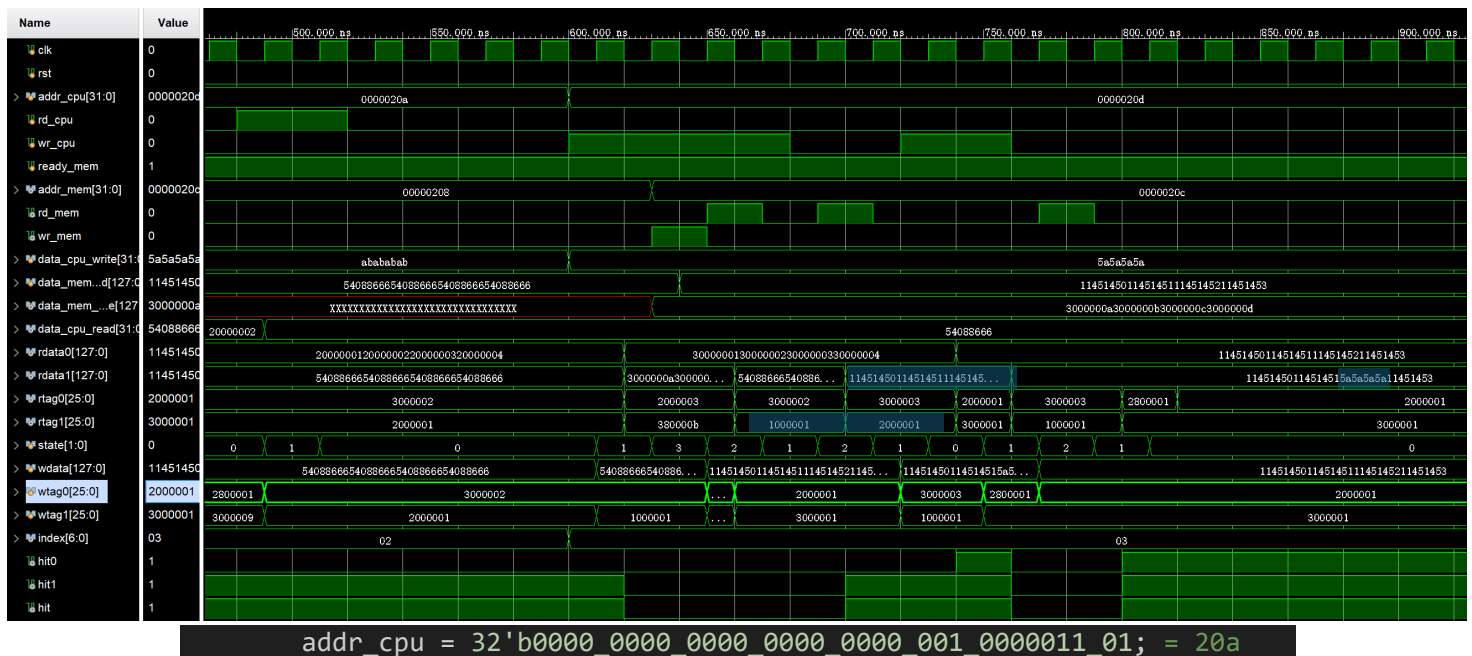
随后对同一地址进行 Read，可以看到 data_cpu_red 成功读出 abababab，即 Read hit。

(3) Read Miss (No dirty): 采用 allocate 从内存中读取块



Index = 0000010 第二组 ——> Tag = 000001 不与 cache 中相应块中的 tag 相等 (miss)
 ——> 从 CompareTag(state = 1) 进入 Allocate (state=2), 因为块 not dirty (rtag 的第 23 位) 不用写回。
 ——> 随后 rdata1 中修改, 且 rtag0 设为 LRU=1 (old) ——> 随后 Read hit

(4) Write Miss (Dirty) (图有点大): 先 WriteBack 写回, 随后 Allocate 从内存中读取块



Index = 0000011 第三组 ——> Tag = 000001 不与 cache 相应块的 tag 相等 (writemiss)
 ——> 从 CompareTag(state = 1) 进入 WB (state=3) (rtag1 块 dirty) 写回。
 ——> 进入 Allocate(state = 2), 并修改 rtag1, 并且 rdata1 写入
 ——> 然后在同一位置写入 5a5a5a5a. 可以看到被修改 (write hit)

```

`timescale 1ns / 1ps // 原代码
module Cache(
    input wire clk, // clock
    input wire rst, // reset
    input wire [31:0] data_cpu_write, // data write in
    input wire [127:0] data_mem_read, // data read in
    input wire [31:0] addr_cpu, // cpu addr
    input wire wr_cpu, // cpu write enable
    input wire rd_cpu, // cpu read enable
    input wire ready_mem, // memory ready

    output reg wr_mem, // memory write enable
    output reg rd_mem, // memory read enable
    output reg [127:0] data_mem_write, // data to mem
    output reg [31:0] data_cpu_read, // data to cpu
    output reg [31:0] addr_mem // memory addr
);

    reg [1:0] state, next_state;
    parameter [1:0] IDLE = 2'b00, CompareTag = 2'b01, Allocate = 2'b10 ,
WriteBack = 2'b11;
    reg en0,en1,ent0,ent1; // input enable
    reg [127:0] wdata; // input din for Data_ram
    reg [25:0] wtag0, wtag1; // input din for Tag_ram
    wire [6:0] index; // input addr for ram
    wire [22:0] cpu_tag; // Tag in addr_cpu
    wire [1:0] cpu_offset;

    wire [127:0] rdata0, rdata1;
    wire [25:0] rtag0, rtag1;

    wire cpu_req_valid;
    wire hit0, hit1, hit;

    wire valid0, valid1, valid;
    wire LRU0, LRU1;
    wire dirty0, dirty1;

    assign cpu_req_valid = wr_cpu | rd_cpu;
    assign index = addr_cpu[8:2];
    assign cpu_tag = addr_cpu[31:9];
    assign cpu_offset = addr_cpu[1:0];
    assign valid0 = rtag0[25];
    assign LRU0 = rtag0[24];

```

```

assign dirty0 = rtag0[23];

assign valid1 = rtag1[25];
assign LRU1 = rtag1[24];
assign dirty1 = rtag1[23];

assign hit0 = (cpu_tag == rtag0[22:0]) & valid0;
assign hit1 = (cpu_tag == rtag1[22:0]) & valid1;
assign hit = hit0 | hit1;
assign valid = valid0 | valid1;
assign dirty = dirty0 | dirty1;

always@(posedge clk or posedge rst) begin
    if(rst) begin
        state <= IDLE;
    end
    else begin
        state <= next_state;
    end
end

always@(*) begin
    case(state)
        IDLE: begin
            en0  = 1'b0;
            en1  = 1'b0;
            ent0 = 1'b0;
            ent1 = 1'b0;
            wr_mem = 1'b0;
            rd_mem = 1'b0;

            if(cpu_req_valid)
                next_state = CompareTag;
            else
                next_state = IDLE;
        end

        CompareTag: begin
            en0  = 1'b0;
            en1  = 1'b0;
            ent0 = 1'b0;
            ent1 = 1'b0;
            wr_mem = 1'b0;
            rd_mem = 1'b0;
        end
    endcase
end

```

```

        if(hit) begin
            next_state = IDLE;
            ent0 = 1'b1;
            ent1 = 1'b1;
            if(rd_cpu) begin
                if(hit0) begin
                    wtag0 = {rtag0[25],1'b0,rtag0[23:0]}; //
change LRU (0 for new)
                    wtag1 = {rtag1[25],1'b1,rtag1[23:0]}; //
change LRU (1 for old)

                    case(cpu_offset)
                        2'b00: data_cpu_read = rdata0[31:0];
                        2'b01: data_cpu_read = rdata0[63:32];
                        2'b10: data_cpu_read = rdata0[95:64];
                        2'b11: data_cpu_read = rdata0[127:96];
                    endcase
                end

                if(hit1) begin
                    wtag0 = {rtag0[25],1'b1,rtag0[23:0]}; //
change LRU (0 for old)
                    wtag1 = {rtag1[25],1'b0,rtag1[23:0]}; //
change LRU (1 for new)

                    case(cpu_offset)
                        2'b00: data_cpu_read = rdata1[31:0];
                        2'b01: data_cpu_read = rdata1[63:32];
                        2'b10: data_cpu_read = rdata1[95:64];
                        2'b11: data_cpu_read = rdata1[127:96];
                    endcase
                end
            end

            if(wr_cpu) begin
                if(hit0) begin
                    en0 = 1'b1;
                    wtag0 = {rtag0[25],1'b0,1'b1,rtag0[22:0]}; //
change LRU (0 for new), change dirty (1 for only in cache)
                    wtag1 = {rtag1[25],1'b1,rtag1[23:0]}; //
change LRU (1 for old)

                    case(cpu_offset)

```

```

                2'b00: wdata =
{rdata0[127:32],data_cpu_write};
                2'b01: wdata =
{rdata0[127:64],data_cpu_write,rdata0[31:0]};
                2'b10: wdata =
{rdata0[127:96],data_cpu_write,rdata0[63:0]};
                2'b11: wdata =
{data_cpu_write,rdata0[95:0]};
            endcase
        end
        else if(hit1) begin
            en1 = 1'b1;
            wtag0 = {rtag0[25],1'b1,rtag0[23:0]}; //
change LRU (1 for old)
            wtag1 = {rtag1[25],1'b0,1'b1,rtag1[23:0]}; //
change LRU (0 for new), change dirty (1 for only in cache)

            case(cpu_offset)
                2'b00: wdata =
{rdata1[127:32],data_cpu_write};
                2'b01: wdata =
{rdata1[127:64],data_cpu_write,rdata1[31:0]};
                2'b10: wdata =
{rdata1[127:96],data_cpu_write,rdata1[63:0]};
                2'b11: wdata =
{data_cpu_write,rdata1[95:0]};
            endcase
        end
    end
end

else begin
    if(dirty)
        next_state = WriteBack;
    else
        next_state = Allocate;
    end
end

Allocate: begin
    en0 = 1'b0;
    en1 = 1'b0;
    ent0 = 1'b0;
    ent1 = 1'b0;

```

```

        rd_mem = 1'b0;
        wr_mem = 1'b0;

        addr_mem = {addr_cpu[31:2],2'b00};
        wdata = data_mem_read;

        if(ready_mem) begin
            next_state = CompareTag;
            rd_mem = 1'b1; // memory read enable
            ent0 = 1'b1;
            ent1 = 1'b1;

            if(LRU0) begin
                en0 = 1'b1;
                wtag0 = {1'b1,1'b0,1'b0,cpu_tag}; // valid , LRU
                // (0 for new), dirty (0 for both in cache and memory)
                wtag1 = {1'b1,1'b1,rtag1[23:0]}; // valid, LRU (1
                // for old)
            end

            else begin
                en1 = 1'b1;
                wtag0 = {1'b1,1'b1,rtag0[23:0]};
                wtag1 = {1'b1,1'b0,1'b0,cpu_tag};
            end

        end

        else
            next_state = Allocate;
    end

    WriteBack: begin
        en0 = 1'b0;
        en1 = 1'b0;
        ent0 = 1'b0;
        ent1 = 1'b0;
        rd_mem = 1'b0;
        wr_mem = 1'b0;
        addr_mem = {addr_cpu[31:2],2'b00};

        if(ready_mem) begin
            next_state = Allocate;
            wr_mem = 1'b1; // memory write enable

```

```

        if(dirty0)
            data_mem_write = rdata0;
        else if(dirty1)
            data_mem_write = rdata1;
        end

        else
            next_state = WriteBack;
        end

        default: next_state = IDLE;
    endcase
end
Data_ram0 data_ram0(
    .clk(~clk),
    .en(en0),//
    .din(wdata),//
    .addr(index),
    .dout(rdata0)//
);
Data_ram1 data_ram1(
    .clk(~clk),
    .en(en1),//
    .din(wdata),//
    .addr(index),
    .dout(rdata1)//
);
Tag_ram0 tag_ram0(
    .clk(~clk),
    .en(ent0),//
    .din(wtag0),//
    .addr(index),
    .dout(rtag0)//
);
Tag_ram1 tag_ram1(
    .clk(~clk),
    .en(ent1),//
    .din(wtag1),//
    .addr(index),
    .dout(rtag1)//
);
endmodule

```